

Masterclass on nonnegative Low-Rank Approximations

Lecture 4 — Automatic music transcription with NMF and CNMF, 5-9 October 2026, Univ. Trento

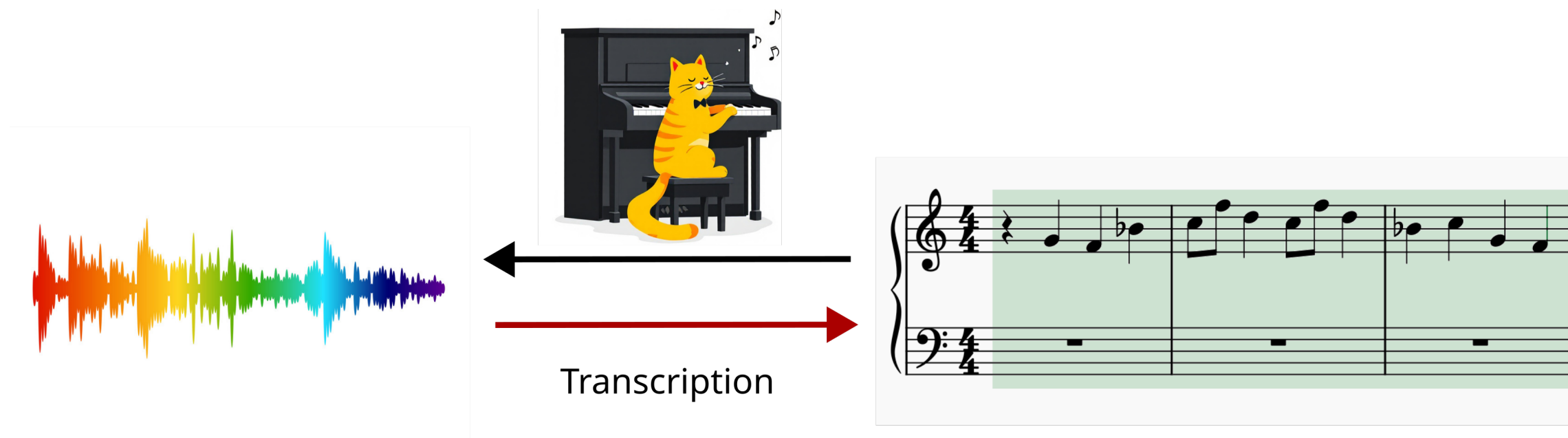
Jérémy E. Cohen
CNRS, CREATIS, Lyon

2026-10-01

Automatic transcription and challenges

What is automatic music transcription?

- **Automatic Music Transcription (AMT)**: convert an audio recording into a symbolic score (MIDI: pitch, onset, offset, velocity) [Benetos2018].
- A long-standing task in music information retrieval.
- Today: piano transcription with **weakly-supervised NMF/CNMF**, joint work with A. Marmoret and H. Wu [Wu2022].



Why AMT is hard

Signal-level difficulties

- Polyphony: many notes overlap in time and frequency
- Timbre varies with instrument, attack, and decay
- Mixing of sources is **non-linear** in the magnitude domain

Model-level difficulties

- Rank / number of notes is unknown
- No uniqueness guarantee for blind low-rank models
- Trained models struggle to generalize to unseen instruments

State of the art and our approach

Supervised deep networks

- Current best performance [Hawthorne2018, Kong2020, Bittner2022]
- Require large labelled datasets
- Black-box, costly to retrain per instrument

Our route: weakly-supervised NMF

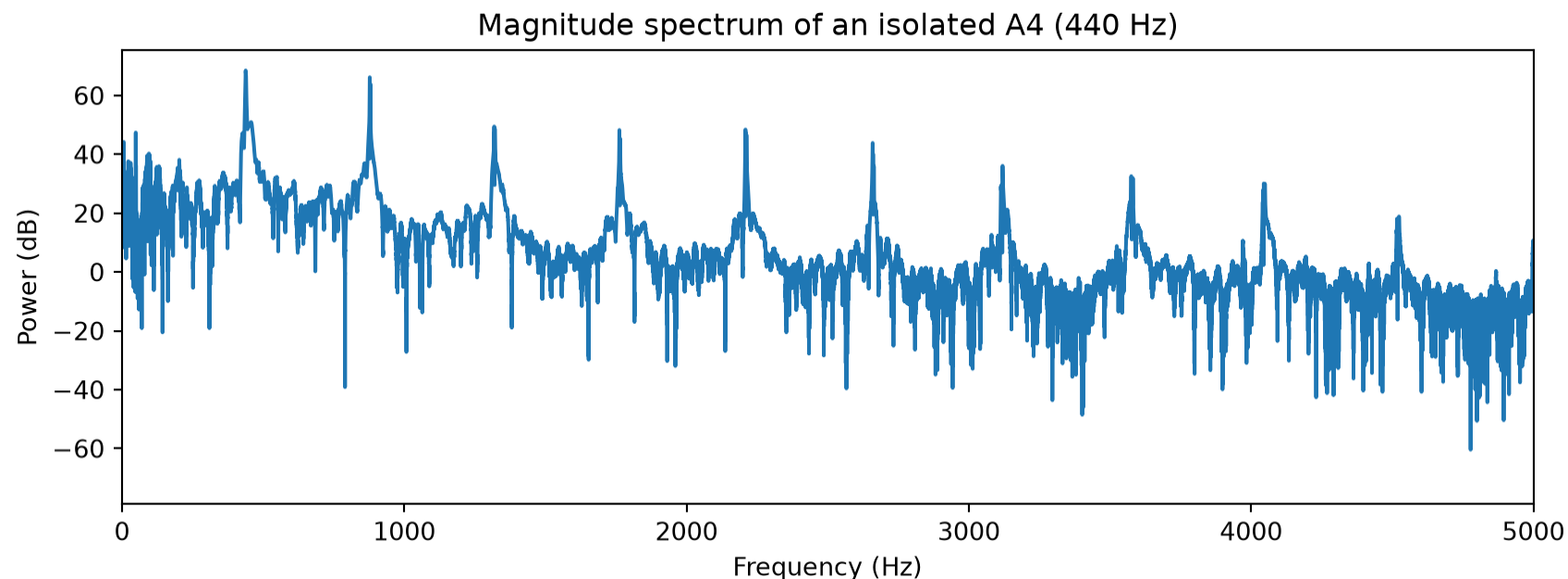
- Exploit that isolated notes are (nearly) low-rank
- Pretrain templates on a handful of isolated notes
- Roadmap: spectrograms $\rightarrow$ low-rank models $\rightarrow$ NMF-KL demo $\rightarrow$ convolutive NMF

Audio and spectrogram representation

A single piano note: comb-shaped spectrum

A piano recording is a time signal $s(t)$. The Fourier transform decomposes it into a sum of pure sinusoids, $\mathcal{F}[s](w) = \int s(t)e^{-2i\pi wt} dt$. Harmonic instruments (piano, strings) produce **comb-shaped spectra**: a fundamental frequency and its harmonics; inharmonic instruments (drums) do not.

```
1 import matplotlib.pyplot as plt
2 import soundfile as sf
3 import numpy as np
4 import scipy.signal as signal
5
6 # Load an isolated A4 (440Hz) note from the MAPS dataset
7 A4_wav, sr = sf.read('../tensorly_hdr/dataset/MAPS_ISOL_LG_M_S0_M69_ENSTDkAm.wav')
8 A4_wav = A4_wav / np.max(np.abs(A4_wav)) # audio normalization
9 A4_wav = A4_wav[:, 0] # use only one channel if stereo
10
11 frequencies_1D = np.fft.fftfreq(len(A4_wav), d=1/sr)
12 # Keep only right part of the spectrum
13 frequencies_1D = frequencies_1D[:len(frequencies_1D)//2]
14 fourier_1D = np.fft.fft(A4_wav)
15 fourier_1D = fourier_1D[:len(fourier_1D)//2]
16 magnitude_1D = np.abs(fourier_1D)
```



From spectrum to spectrogram: the STFT



0:00 / 0:13



A4 (440 Hz), MAPS dataset [Emiya2010]

- One spectrum describes the whole ~15s recording
- But the note is not stationary: attack, decay, noise at the end
- We need a **time-varying** spectral representation

The Short-Time Fourier Transform (STFT) slides a window over the signal and computes a spectrum for each time frame:

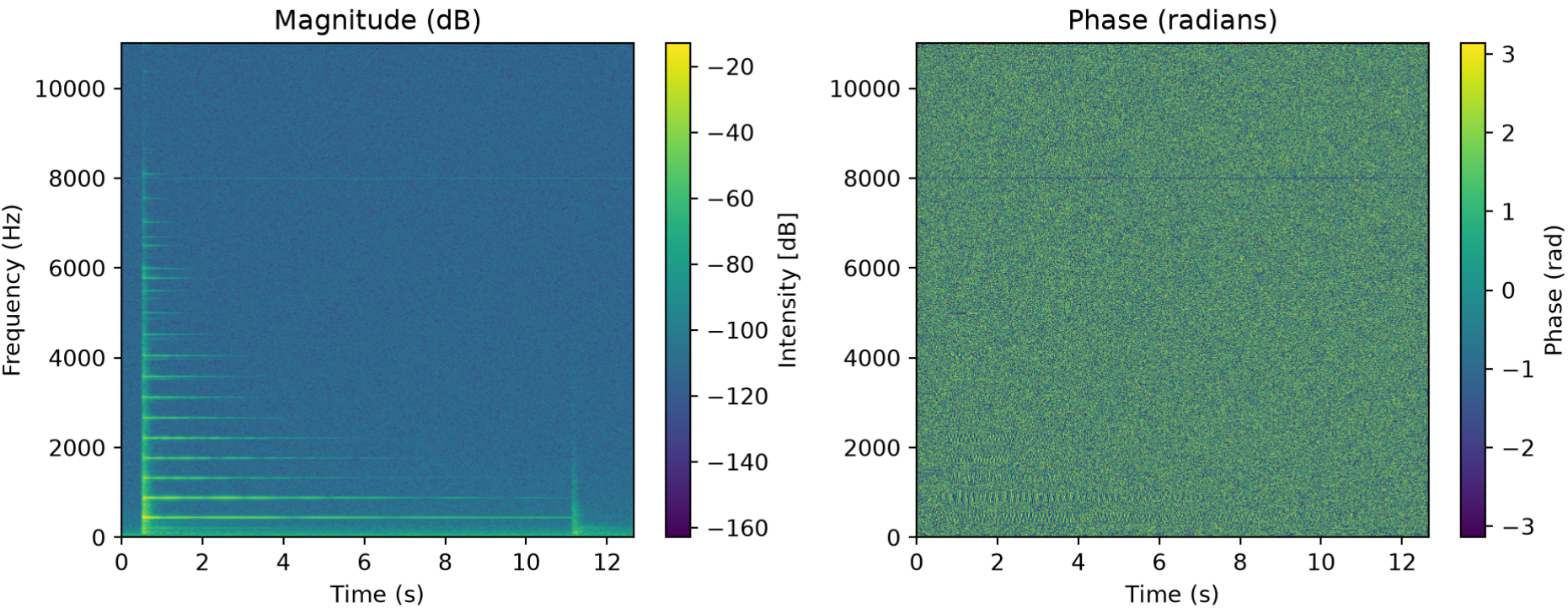
$$Y[f, t] = \sum_{\tau} s(\tau) w(\tau - t) e^{-2i\pi f\tau}.$$

We use Hann windows of 4096 samples (~93ms) with heavy overlap (20ms hop), a standard trade-off for piano transcription:

```
1 frequencies, times, Sxx = signal.stft(A4_wav, fs=sr, nperseg=4096, nfft=4096, noverlap=4096 - 882)
2 half_freqs = len(frequencies) // 2
3 frequencies = frequencies[:half_freqs]
4 Sxx = Sxx[:half_freqs, :] # keep only lower half
5 Sxx_abs = np.abs(Sxx)
6 Sphase = np.angle(Sxx)
7 Sxx_dB = 20 * np.log10(Sxx_abs + 1e-10)
```


Magnitude and phase spectrograms

For AMT we typically only use the **magnitude** spectrogram below; the phase (right) is much harder to interpret and to model.



Frobenius and KL low-rank approximations

Best rank-one approximation: two losses

The magnitude spectrogram of a single note is well approximated by a **rank-one matrix**: a fixed frequency profile (the comb) times a single time envelope (attack-decay). This motivates using low-rank NMF to analyse recordings with several notes.

Frobenius (Eckart-Young)

$$\min_{u,v} \|Y - uv^T\|_F^2$$

closed form via the truncated SVD: $u = U[:, 1]$,
 $v = S[1, 1]V[:, 1]$.

Kullback-Leibler

$$\min_{u,v \geq 0} D_{\text{KL}}(Y, uv^T)$$

closed form via normalized marginals:

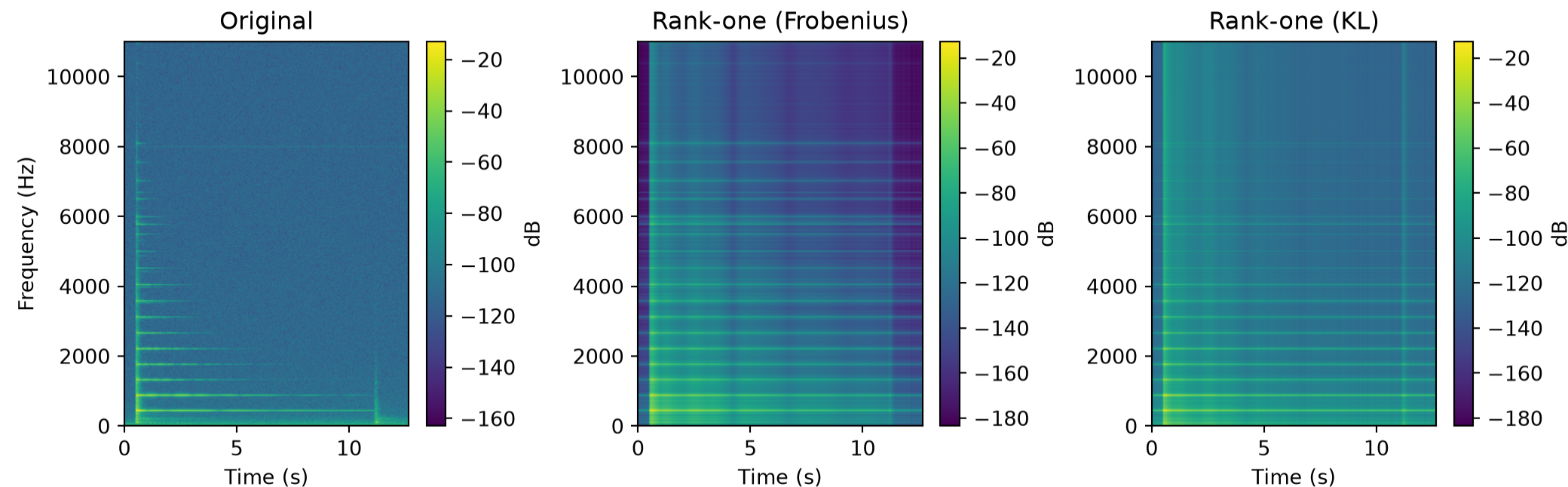
$$u = \frac{\sum_t Y[:, t]}{\sqrt{\sum_{f,t} Y[f, t]}}, \quad v = \frac{\sum_f Y[f, :]}{\sqrt{\sum_{f,t} Y[f, t]}}.$$

Computing the two approximations

We apply both closed-form formulas to the A4 magnitude spectrogram: the SVD gives the Frobenius-optimal rank-one factors, while row/column sums give the KL-optimal ones.

```
1 # Best rank-one approximation of the magnitude spectrogram, Frobenius sense
2 U, s, V = np.linalg.svd(Sxx_abs)
3 r1_approx = s[0] * np.outer(U[:, 0], V[0, :])
4
5 # Best rank-one approximation, KL sense (normalized marginals)
6 ukl = np.sum(Sxx_abs, axis=1) / np.sqrt(np.sum(Sxx_abs))
7 vkl = np.sum(Sxx_abs, axis=0) / np.sqrt(np.sum(Sxx_abs))
8 r1_approx_kl = np.outer(ukl, vkl)
```

Frobenius vs KL rank-one approximation



Both rank-one models capture the comb-shaped spectrum well; the KL version tends to spread energy more smoothly in time.

From one note to many: additive mixing

Two simultaneous notes with time signals $s_1(t)$, $s_2(t)$ are recorded as $s(t) = s_1(t) + s_2(t)$. By linearity of the Fourier transform, $\mathcal{F}[s](w) = \mathcal{F}[s_1](w) + \mathcal{F}[s_2](w)$, but we only observe the **magnitude** $|\mathcal{F}[s](w)|$. If each $|\mathcal{F}[s_i](w)|$ is rank-one, is $|\mathcal{F}[s](w)|$ rank-two?

$$|\mathcal{F}[s](w)|^2 = |\mathcal{F}[s_1](w)|^2 + |\mathcal{F}[s_2](w)|^2 + 2 \operatorname{Re}(\mathcal{F}[s_1]^*(w) \mathcal{F}[s_2](w))$$

Important

When is the magnitude spectrogram rank-two? Mostly when $|\mathcal{F}[s](w)| = |\mathcal{F}[s_1](w)| + |\mathcal{F}[s_2](w)|$

We can study this using the polar representation of the spectra

$$\mathcal{F}[s_1](w) = A_1(w)e^{i\theta_1(w)}, \quad \mathcal{F}[s_2](w) = A_2(w)e^{i\theta_2(w)}$$

$$|\mathcal{F}[s](w)| = \sqrt{A_1(w)^2 + A_2(w)^2 + 2A_1(w)A_2(w)\cos(\theta_2(w) - \theta_1(w))}$$

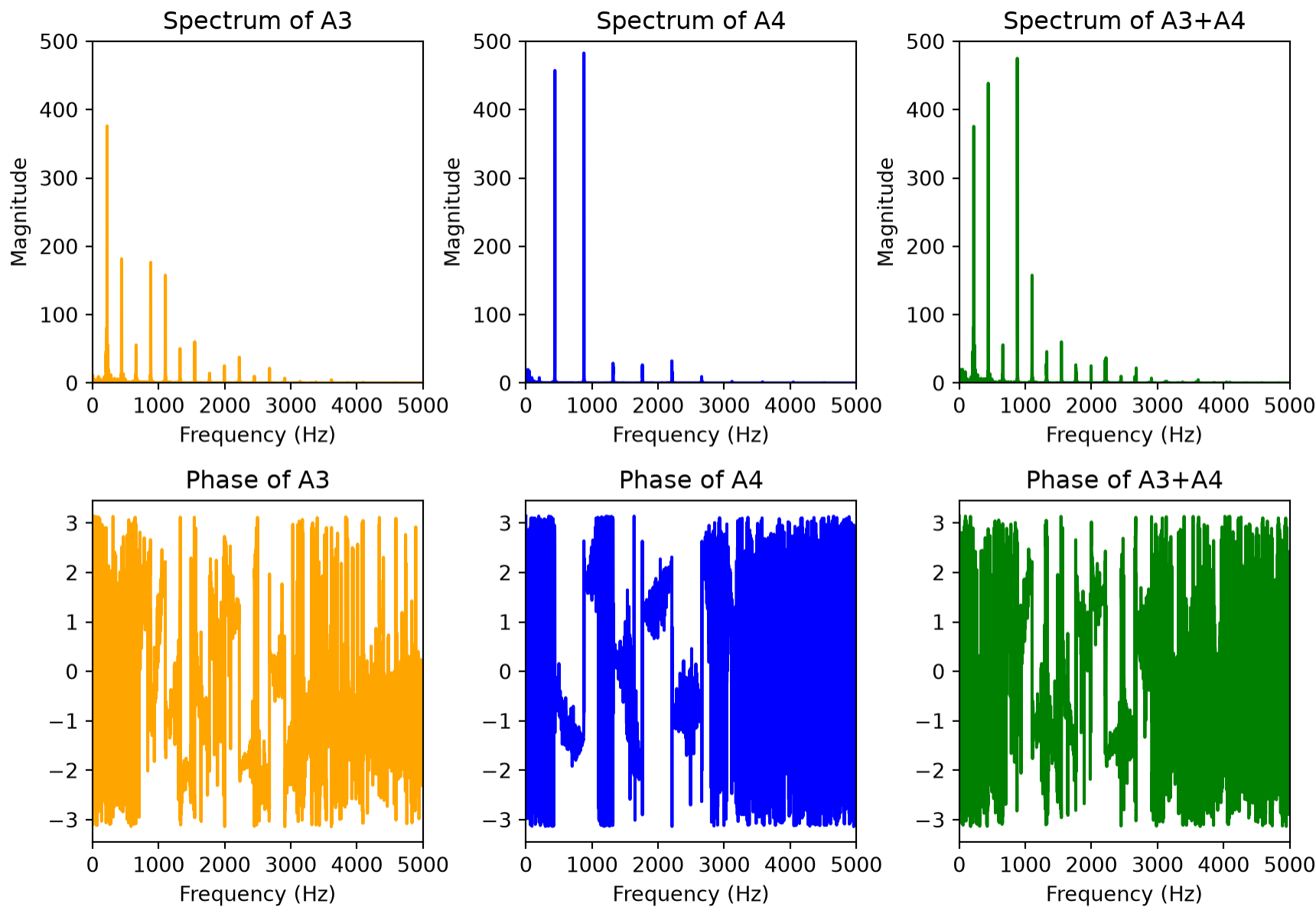
When is the mixture exactly rank-two?

- A sufficient case: the two signals are **in phase**, $\theta_1 = \theta_2$
 - We get $\sqrt{A_1^2 + A_2^2 + 2A_1A_2 \cos(\theta_2(w) - \theta_1(w))} = \sqrt{A_1^2 + A_2^2 + 2A_1A_2} = \sqrt{(A_1 + A_2)^2}$.
- Another sufficient case: the two amplitudes have **disjoint support**, $A_1(w)A_2(w) = 0$.
 - Also works for the power spectrogram $|\mathcal{F}[s](w)|^2$.
 - This is what we use in general, as disjoint supports are more likely than in-phase signals.
 - In practice: two notes an **octave apart** share many partials, and the disjoint support condition is violated
 - $\Rightarrow$ low-rank models of spectrograms are prone to **octave errors**

Phase interference

A3 and A4 played together

0:00 / 0:13 0:00 / 0:15



Sum of magnitudes vs magnitude of sum

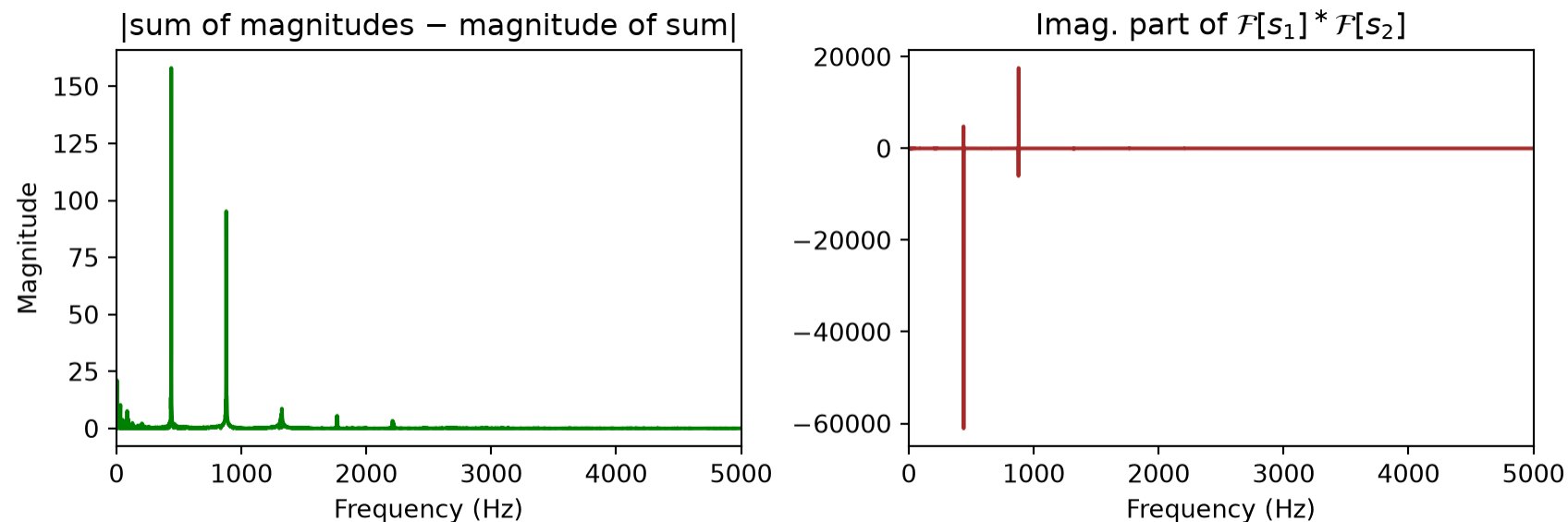
We compare the sum of amplitudes to the amplitude of the sum on a short segment of each recording.

To test the support condition from the previous section, we take a one-second rectangular-window segment of each note and compare their individual and combined Fourier transforms.

```
1 A3_wav, sr = sf.read('../tensorly_hdr/dataset/MAPS_ISOL_LG_F_S1_M57_ENSTDkAm.wav')
2 A3_wav = A3_wav / np.max(np.abs(A3_wav))
3 A3_wav = A3_wav[:, 0]
4
5 # Short rectangular-window segment for both notes
6 t_1, t_2 = 2 * sr, 3 * sr
7 A4_segment = A4_wav[t_1:t_2]
8 A3_segment = A3_wav[t_1:t_2]
9
10 freq_1D = np.fft.fftfreq(len(A4_segment), d=1/sr)
11 freq_1D = freq_1D[:len(freq_1D)//2]
12 fft_A4 = np.fft.fft(A4_segment)
13 fft_A4 = fft_A4[:len(fft_A4)//2]
14 fft_A3 = np.fft.fft(A3_segment)
15 fft_A3 = fft_A3[:len(fft_A3)//2]
16 mag_A4, mag_A3 = np.abs(fft_A4), np.abs(fft_A3)
17
18 magnitude_of_sum = np.abs(fft_A4 + fft_A3)
19 sum_of_magnitudes = mag_A4 + mag_A3
```

Where does the mismatch come from?

The two quantities should coincide if the notes were perfectly support. The residual (left) reveals where they are not, and it matches exactly the imaginary part of the cross-spectrum (right), the phase-mismatch term from the Cauchy-Schwarz expansion.



A3 and A4 are **not** orthogonal: the mismatch is largest exactly at their shared partials.

Consequences

- Two notes an octave apart share fundamental and harmonics in frequency, but **not in phase**
- The colinearity assumption behind rank-additivity fails at shared partials
- Low-rank models (NMF included) are structurally prone to confusing octaves
- This is a **modeling** limitation, not just an optimization one

Simple-piece live code demo

The recording: isolated notes then chords



0:00 / 0:10



MAPS recording: six isolated notes, then two five-note chords [Emiya2010]

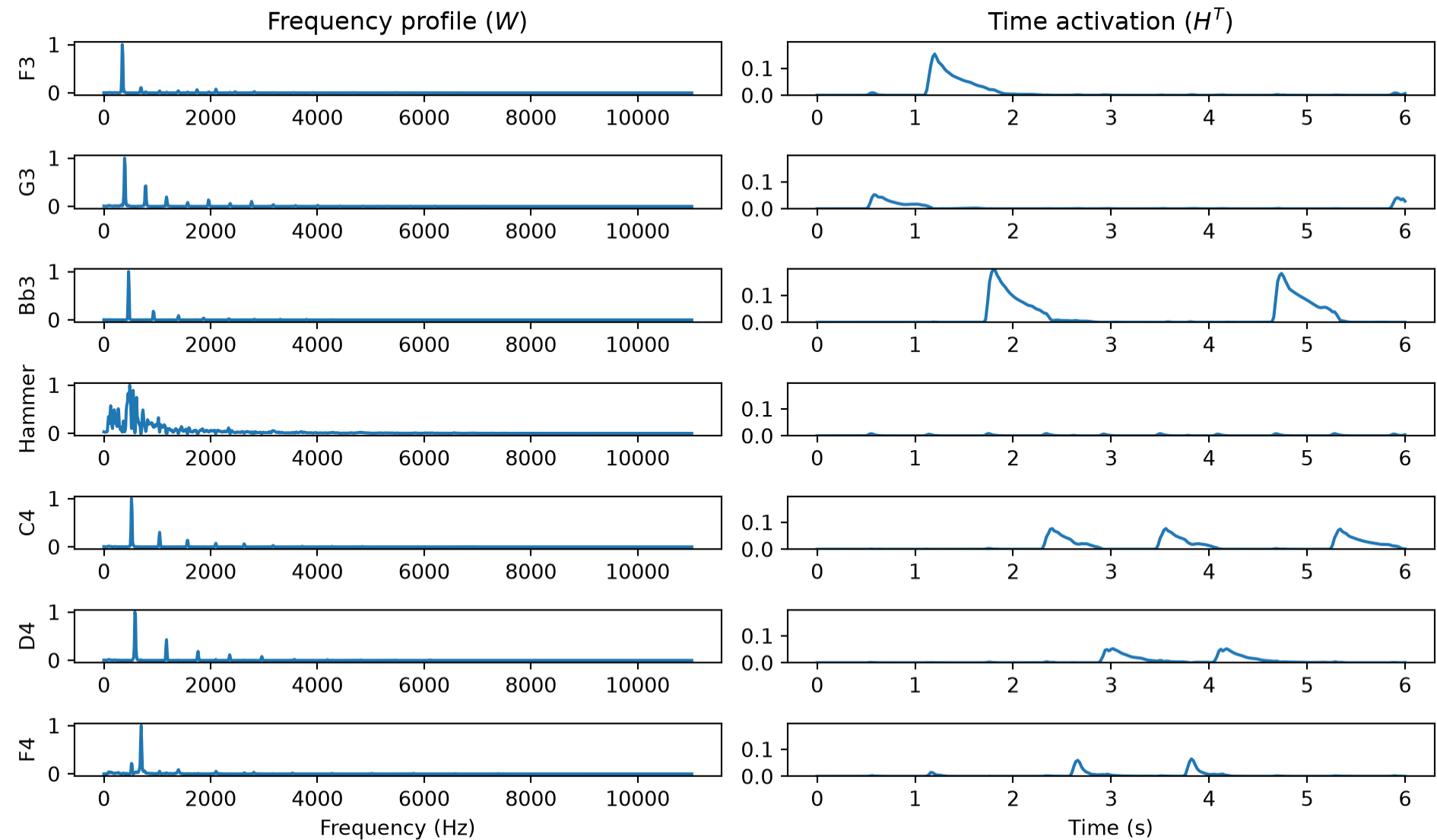


Computing the spectrogram, then fitting NMF-KL

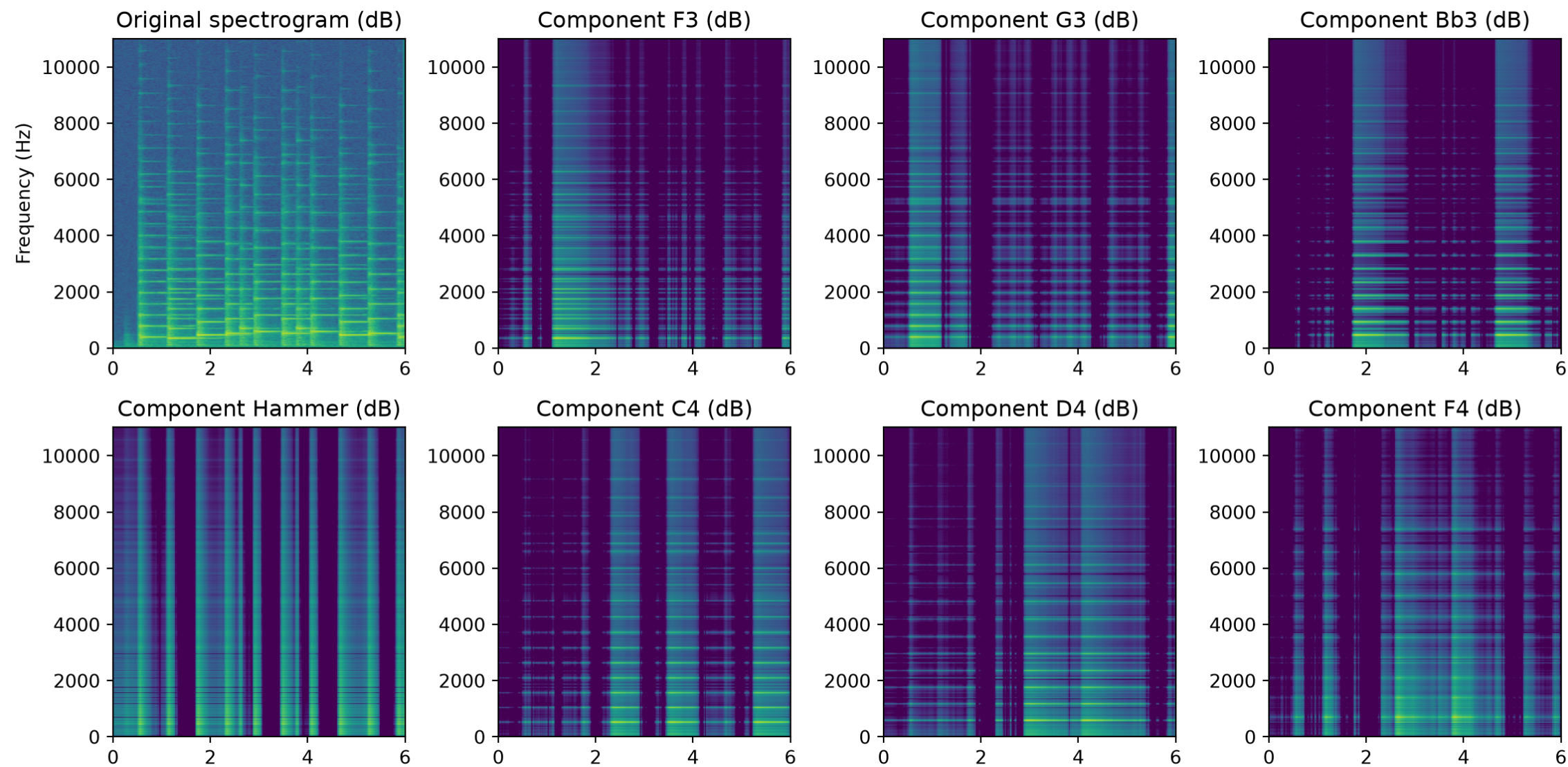
- We first cut the recording to its first six seconds (six isolated notes only) and compute its magnitude spectrogram.
- We then fit an NMF with the KL divergence using the multiplicative-update algorithm, with a rank of 7: six notes plus one extra component expected to capture the hammer action.

```
1 t_1, t_2 = 0, 6 * sr # first six seconds: isolated notes only
2 song_simple = song[t_1:t_2]
3 frequencies, times, Sxx = signal.stft(song_simple, fs=sr, nperseg=4096, nfft=4096, noverlap=4096 - 882)
4 half_freqs = len(frequencies) // 2
5 frequencies = frequencies[:half_freqs]
6 Sxx = Sxx[:half_freqs, :]
7 Sxx_dB = 20 * np.log10(np.abs(Sxx) + 1e-10)
8 Sxx_abs = np.abs(Sxx)
9
10 from tensorly_hdr.nmf_kl import Lee_Seung_KL
11 from tensorly_hdr.image_utils import permute_spectra
12
13 rank = 7
14 np.random.seed(0)
15 W_init = np.abs(np.random.randn(Sxx.shape[0], rank))
16 H_init = np.abs(np.random.randn(Sxx.shape[1], rank)).T
17 crit, W_kl, H_kl, toc, cnt = Lee_Seung_KL(Sxx_abs, W_init, H_init, NbIter=20, verbose=False, print_it=20)
18
19 # Normalize W columnwise, put the norm in H, order by increasing fundamental frequency
20 W_kl_norms = np.max(W_kl, axis=0)
21 W_kl = W_kl / W_kl_norms
22 H_kl = H_kl.T * W_kl_norms
23 perm = permute_spectra(W_kl)
24 W_kl, H_kl = W_kl[:, perm], H_kl[:, perm]
25 notes = ["F3", "G3", "Bb3", "Hammer", "C4", "D4", "F4"]
```

Frequency profiles and time activations



Rank-one KL reconstructions per note



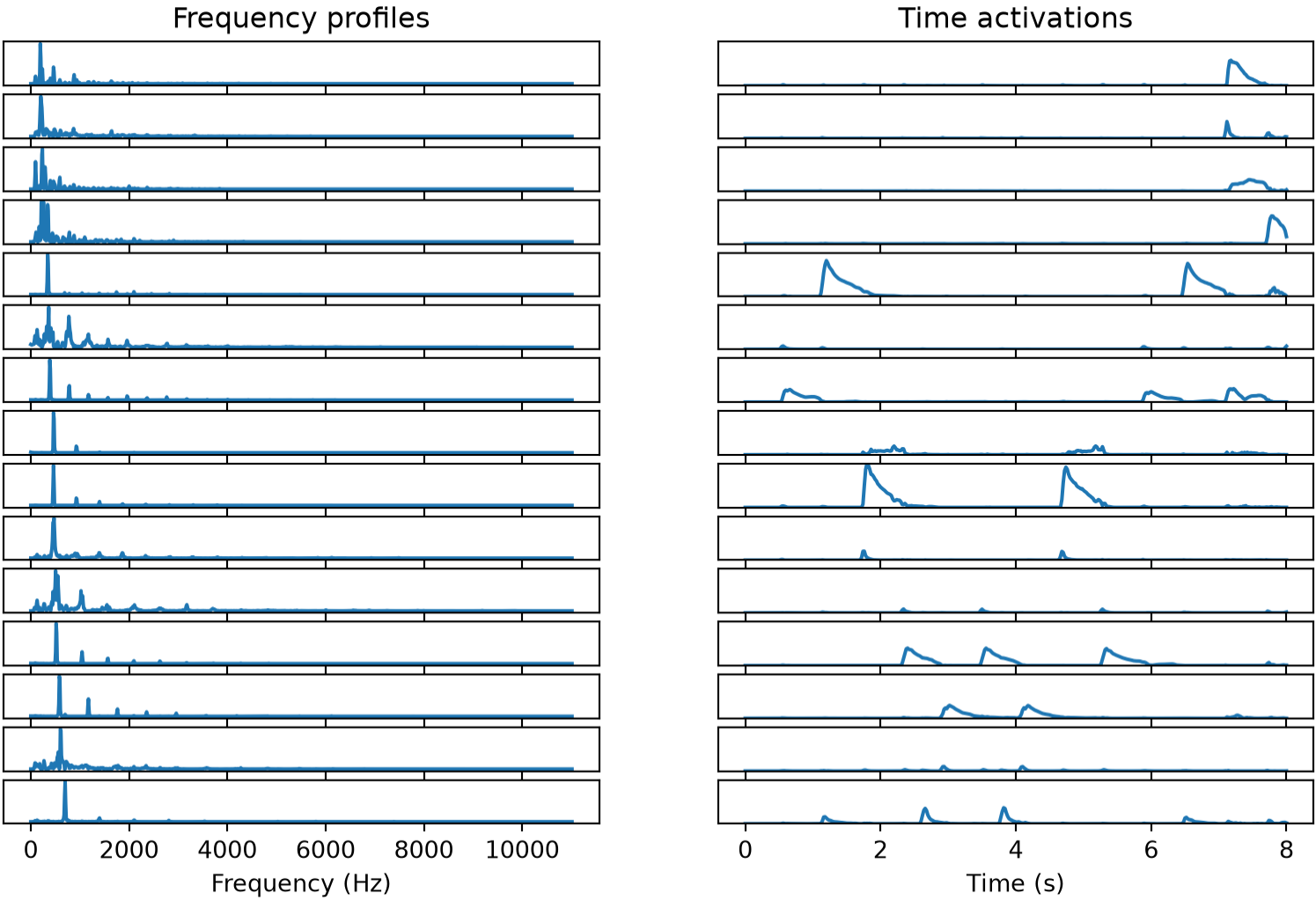
Six isolated notes are recovered almost perfectly, plus a “hammer” component capturing the percussive attack.

Adding the chords: rank 15

We now extend the excerpt to eight seconds, adding the two five-note chords (14 distinct notes in total), and refit NMF-KL with 15 components.

```
1 t_1, t_2 = 0, 8 * sr # add the two five-note chords
2 song_full = song[t_1:t_2]
3 frequencies, times, Sxx = signal.stft(song_full, fs=sr, nperseg=4096, nfft=4096, noverlap=4096 - 882)
4 half_freqs = len(frequencies) // 2
5 frequencies = frequencies[:half_freqs]
6 Sxx = Sxx[:half_freqs, :]
7 Sxx_abs = np.abs(Sxx)
8
9 rank = 15
10 W_init = np.abs(np.random.randn(Sxx.shape[0], rank))
11 H_init = np.abs(np.random.randn(Sxx.shape[1], rank)).T
12 crit, W_kl15, H_kl15, toc, cnt = Lee_Seung_KL(Sxx_abs, W_init, H_init, NbIter=20, verbose=False, print_it=20)
13 W_kl15_norms = np.max(W_kl15, axis=0)
14 W_kl15 = W_kl15 / W_kl15_norms
15 H_kl15 = H_kl15.T * W_kl15_norms
16 perm = permute_spectra(W_kl15)
17 W_kl15, H_kl15 = W_kl15[:, perm], H_kl15[:, perm]
```

Adding the chords: rank 15



NMF struggles with overlapping chords

- There are 14 distinct notes in the two chords, but NMF-KL does **not** recover 14 clean components
- Several components refine already-identified notes instead of finding new ones
- Low-fundamental components are hardly interpretable, and their activations do not follow an attack-decay pattern
- This questions both the **rank choice** and the **uniqueness** of the decomposition

Problems

- **Model mismatch:** the rank-one hypothesis for a single note is only approximate, so NMF spends extra components on it rather than on new notes
- **Identifiability:** no guarantee that output spectra stay comb-shaped; uniqueness is unclear as soon as notes overlap, the rule with chords

Attempted fixes

- Standard pipeline: pick the fundamental frequency of each rank-one component, threshold its activation [Vincent2008]
- Parametrize activations to follow an attack-decay pattern [Cheng2016]
- We instead enrich the *model* itself, not just the post-processing

Our route: convolutive NMF + weak supervision

Two ideas combined [Wu2022]

1. Replace the rank-one note model by a **convolutive** rank-one model (better fit to attack/decay)
2. **Pretrain** the templates on isolated note recordings (weak supervision) instead of learning them blindly

CNMF model, algorithm, and results

Rank-one convolutive model

A single note spectrogram is modeled as a convolution of a small time-frequency template with a time activation:

$$\hat{Y}[f, t] = \sum_{\tau=0}^{T-1} W[f, \tau] h[t - \tau],$$

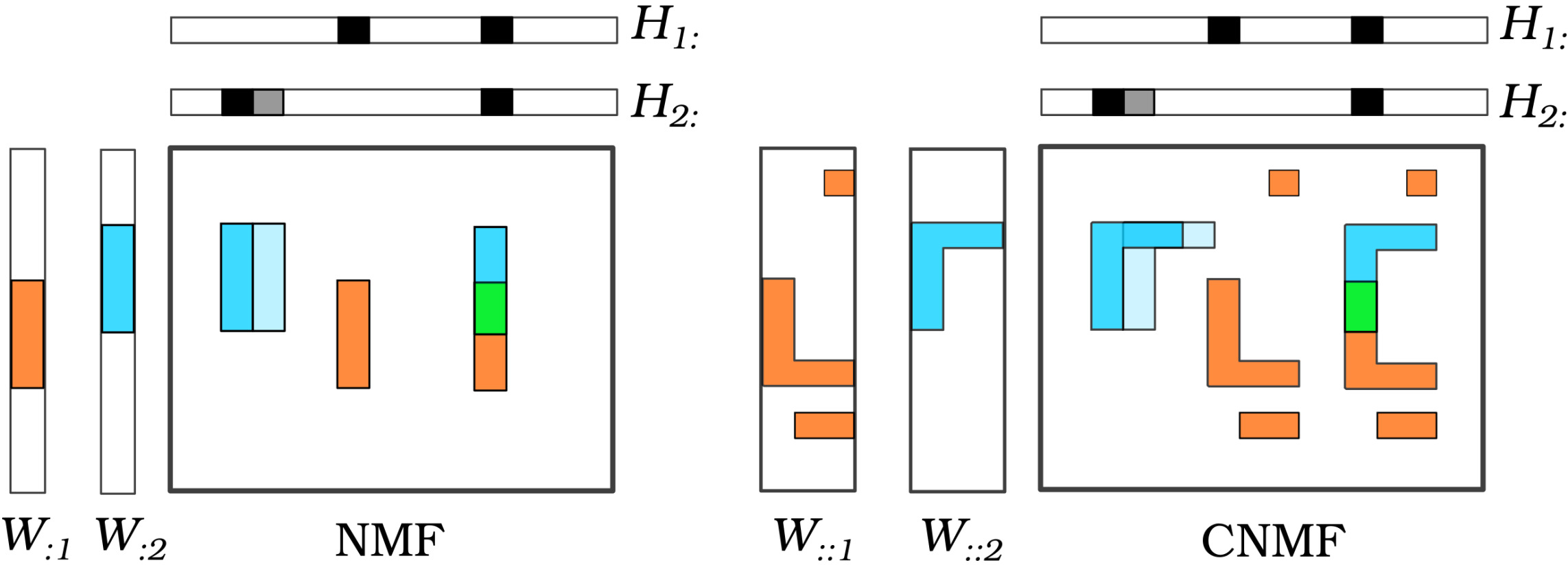
where T is the convolution window length (captures attack + early decay), fixing the main weakness of the plain rank-one model. Stacking shifted copies of h into a Toeplitz matrix $\tilde{H}$, this is a constrained rank- T NMF:

$$\hat{Y} = W\tilde{H}.$$

The full **Convolutional NMF** [Smaragdis2006] sums such terms over r components:

$$\hat{Y} = \sum_{q=1}^r \sum_{\tau=0}^{T-1} W[f, \tau, q] H[q, t - \tau].$$

NMF vs CNMF visually



NMF vs CNMF multiplicative updates

Following the MM2 algorithm of [Fagot2019], with $\hat{Y}$ the current reconstruction:

$$W^{(k+1)} = W^{(k)} * \frac{\frac{Y}{\hat{Y}^{(k)}} \tilde{H}^{(k)T}}{\mathbf{1} \otimes \sum_t \tilde{H}^{(k)}[:, t]},$$
$$H^{(k+1)}[:, t] = H^{(k)}[:, t] * \frac{W^{(k)} \times_{1,2} \frac{Y}{\hat{Y}}[:, t : t + T]}{\sum_{f, \tau} W^{(k)}[f, \tau, :] \mathbf{1}_{t+\tau \leq n}}.$$

Same majorization principle as plain NMF; CNMF is harder to optimize in practice, with many non-trivial local minima.

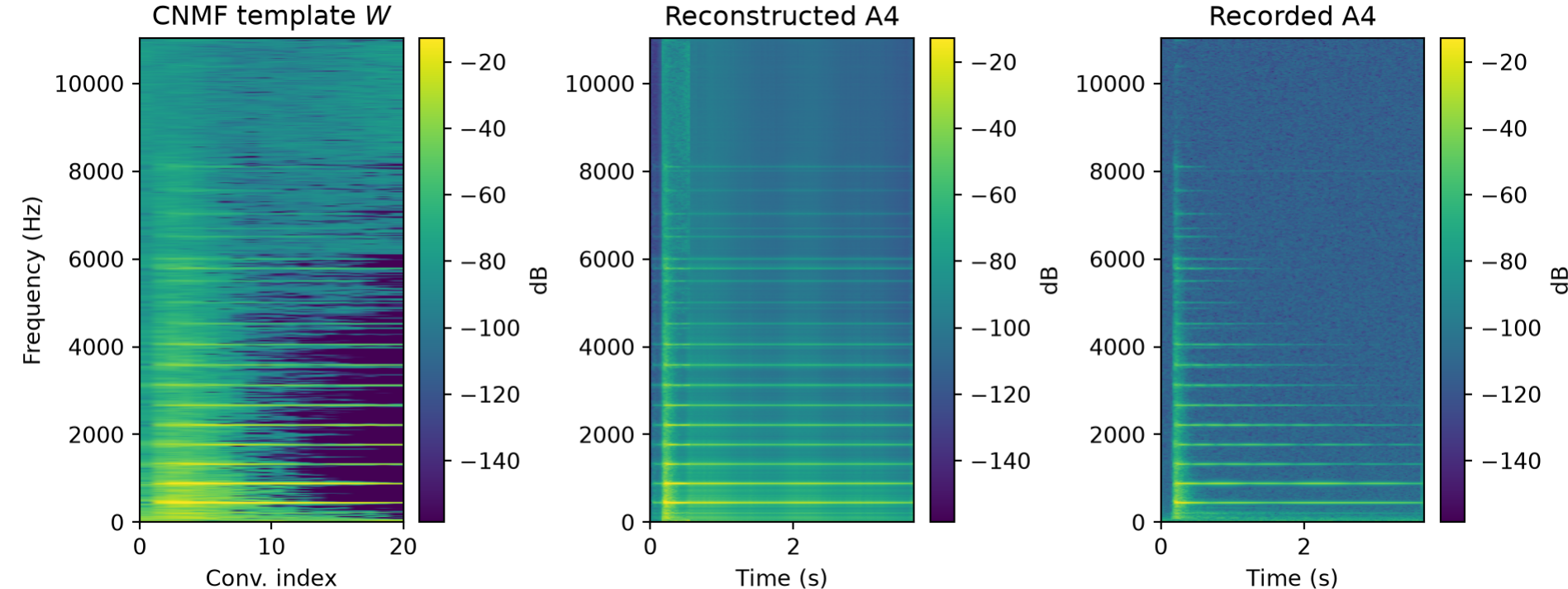
Rank-one CNMF on A4

We now pretrain templates on isolated notes: MAPS provides an isolated recording for every note we need, and for each one we fit a **rank-one CNMF** to obtain a template W . Per-note templates will later be stacked into a tensor

$W \in \mathbb{R}_+^{f \times T \times r}$ and used to transcribe full recordings. We start with A4:

```
1 from tensorly_hdr.cnmf import convolutive_nmf, shift_m
2
3 A4_wav_train = A4_wav[sr // 3: sr * 4] # avoid the noisy tail of the recording
4 frequencies, times, Sxx = signal.stft(A4_wav_train, fs=sr, nperseg=4096, nfft=4096, noverlap=4096 - 882)
5 half_freqs = len(frequencies) // 2
6 Sxx_dB = 20 * np.log10(np.abs(Sxx[:half_freqs, :]) + 1e-10)
7 Sxx_abs = np.abs(Sxx[:half_freqs, :])
8 Sxx_abs = Sxx_abs / np.max(Sxx_abs)
9
10 T = 20
11 W_r1, H_r1, err_r1 = convolutive_nmf(
12     Sxx_abs, rank=1, T=T, itmax=10, eps=1e-16, tol=0,
13     print_it=1, n_iter_inner=10, init="separable", verbose=False
14 )
15 W_cnmf, h_cnmf = W_r1[:, :, 0], H_r1[0, :]
```

Template vs reconstructed spectrogram

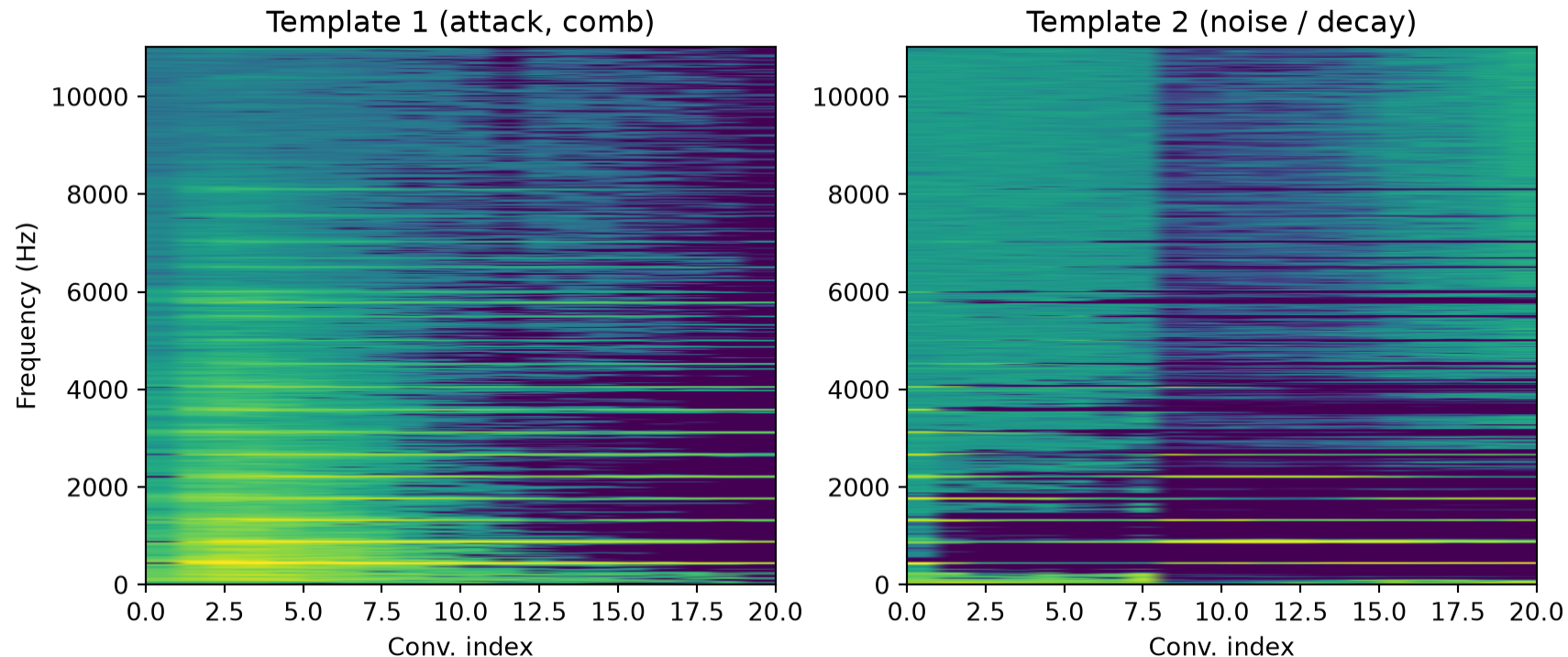


The attack is well captured, but high-frequency content after the attack is not fully realistic (echoes in the template).

Rank-two CNMF cleans the template

A second, randomly-initialized component can absorb this echo/noise, leaving the first template with a cleaner comb spectrum:

```
1 W_r2, H_r2, err_r2 = convlutive_nmf(  
2     Sxx_abs, rank=2, T=T, itmax=10, eps=1e-16, tol=0,  
3     print_it=1, n_iter_inner=10, init="separable", verbose=False  
4 )
```



For speed, we keep **rank-one** CNMF per note in the pipeline below.

Learning 14 note templates

We repeat the rank-one CNMF fit for every isolated note in the `notes_to_learn` folder, and stack the resulting templates into a tensor W ordered by increasing MIDI pitch:

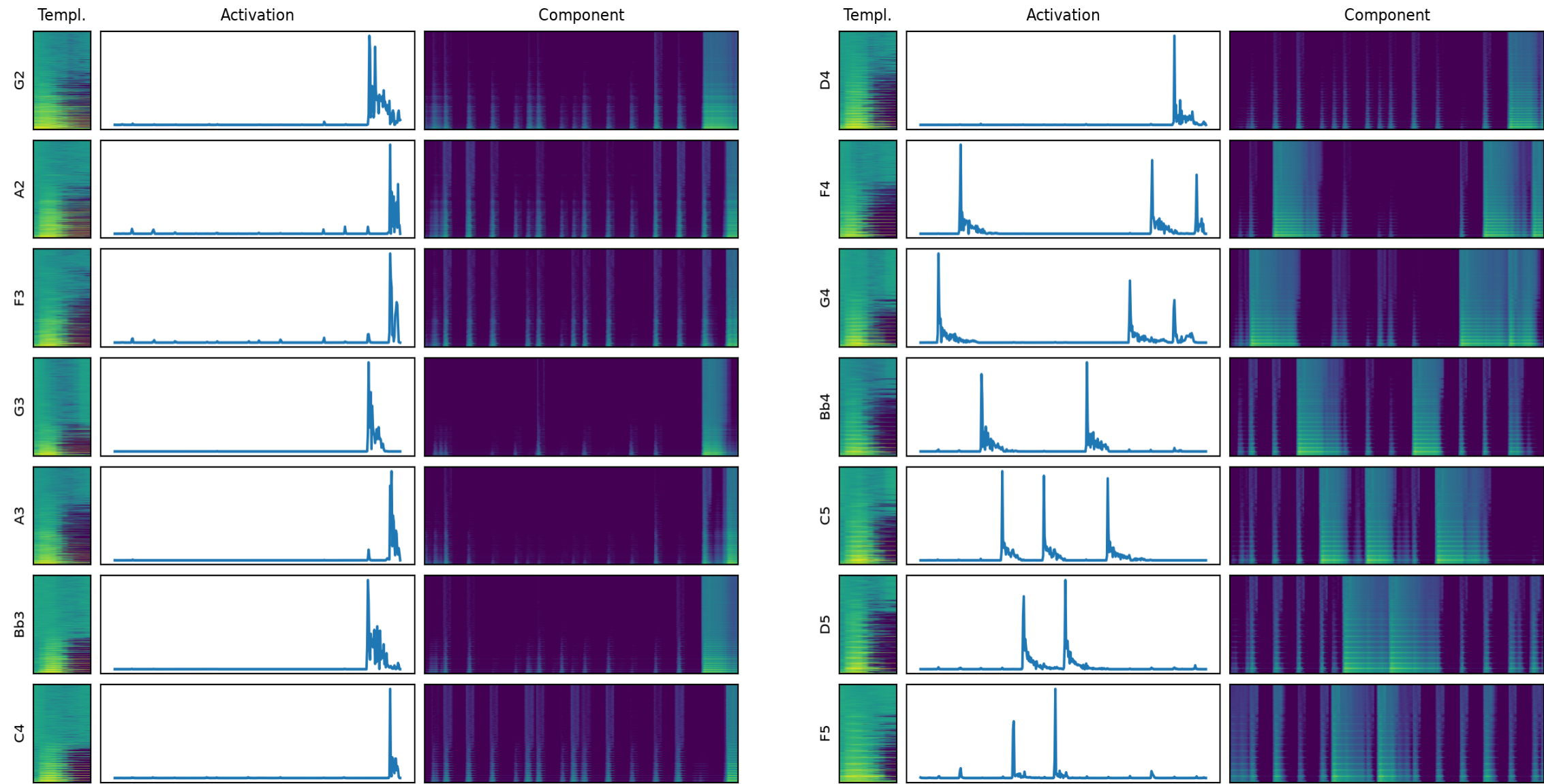
```
1 import os
2 import re
3 from tensorly_hdr.cnmf import convolutive_nmf
4
5 data_dir = '../..'/tensorly_hdr/dataset/notes_to_learn'
6 T = 10
7 Wlist, names = [], []
8 for filename in os.listdir(data_dir):
9     names.append(re.search(r'M(\d+)_', filename).group(1))
10    wav, sr = sf.read(os.path.join(data_dir, filename))
11    wav = wav / np.max(np.abs(wav))
12    wav = wav[:, 0][0:sr * 4]
13    freqs, _, S = signal.stft(wav, fs=sr, nperseg=4096, nfft=4096, noverlap=4096 - 882)
14    S = S[:, :-2]
15    half = len(freqs) // 2
16    S_abs = np.abs(S[:half, :])
17    S_abs = S_abs / np.max(S_abs)
18    Wl, Hl, _ = convolutive_nmf(S_abs, rank=1, T=T, itmax=10, eps=1e-16, tol=0, print_it=10, n_iter_inner=10, init="separable", verbose=False)
19    Wlist.append(Wl[:, :, 0])
20
21 rank = len(Wlist)
22 W = np.stack(Wlist, axis=2)
23 perm = np.argsort(names)
24 W = W[:, :, perm]
25 names_music = ["G2", "A2", "F3", "G3", "A3", "Bb3", "C4", "D4", "F4", "G4", "Bb4", "C5", "D5", "F5"]
```

Transcribing by convolutive regression

With templates W fixed, transcribing the 8-second excerpt (six notes + two chords) now only requires estimating the activations H , by convolutive regression against the fixed dictionary:

```
1 from tensorly_hdr.cnmf import convolutive_regression
2
3 t_1, t_2 = 0, 8 * sr
4 song_tr = song[t_1:t_2]
5 frequencies, times, Sxx = signal.stft(song_tr, fs=sr, nperseg=4096, nfft=4096, noverlap=4096 - 882)
6 Sxx = Sxx[:, :-2]
7 times = times[:-2]
8 half_freqs = len(frequencies) // 2
9 frequencies = frequencies[:half_freqs]
10 Sxx = Sxx[:half_freqs, :]
11 Sxx_dB = 20 * np.log10(np.abs(Sxx) + 1e-10)
12 Sxx_abs = np.abs(Sxx)
13
14 H_tr, err = convolutive_regression(Sxx_abs, W, itmax=50, verbose=False, print_it=25)
```

Transcription result



The two five-note chords are correctly identified, with imperfect but plausible relative activations.

Transcription performance

F-measure (%) at 50ms tolerance, per piano [Wu2022]:

Method	EN1	EN2	AkB1	AkB2	AkC	AkS
CNMF	88	80	95	90	94	90
AD [Cheng2016]	82	69	68	73	61	69
DNN [Kong2020]	89	77	98	95	98	87

CNMF with weak supervision reaches performance close to fully supervised deep networks.

Note-wise **precision** = $TP / (TP + FP)$, **recall** = $TP / (TP + FN)$, **F-measure** = $2 PR / (P + R)$ (TP/FP/FN counted at 50ms tolerance on onsets).

Generalization

Train \ Test	AkC	EN1	EN2	AkB1	AkB2
AkC	94	74	74	77	77
EN1	74	88	76	67	68

Performance drops sharply across pianos: the pipeline is **not robust to distribution shift**.

Takeaways and research directions

- Single-note spectrograms are nearly low-rank, but the additive mixture is not exactly low-rank: phase interference and octave errors are structural, not just numerical
- Plain NMF-KL is a good first analysis tool, but lacks a reliable rank / uniqueness story for chords
- Convolutional NMF with pretrained templates recovers competitive transcriptions with **weak supervision only**
- Open problems: robustness to instrument/recording shift, joint source separation and AMT — part of my ongoing research project